CANONICALIZATION FOR DNA FOUNDATIONAL MODELS: A PROOF-OF-CONCEPT

Felipe Verri
pipe.verri@gmail.com

ABSTRACT

A DNA sequence and it's reverse-complement(RC) represent the same molecule, and a model that cannot recognize this RC-symmetry wastes capacity relearning the same biological signal on both sequences. Yet, the two strands are not always functionally interchangeable: There are certain asymmetries introduced by transcription that a model can only exploit if it is shown in a consistent orientation with respect to the template and coding strands, rather than an arbitrary one. We test canonicalization, fixing which physical strand (coding or template) occupies each input position, derived from genome annotation (GFF) files, as a way to let a symmetry-aware but not fully RC-invariant/equivariant architecture (ConvNova) make use of these asymmetries. We evaluate this proof of concept on 30 tasks from multiple benchmarks comparing against both a non-canonicalized baseline and a leakage-control model that receives the same strand information as an explicit input channel, since the annotation-derived orientation signal used for canonicalization can itself leak information. Canonicalization, as implemented here, fails: across the tasks selected for full-scale evaluation, it does not outperform the leakage control model, who does show a consistent improvement over the plain baseline on most tasks. That is at least compatible with the idea that transcriptional-asymmetry information can be useful to these models ; this is only suggestive, not evidence for canonicalization itself, and the effect could equally stem from other factors. We conclude that this particular canonicalization approach does not work, while whether exploiting transcriptional asymmetry through some other mechanism could work remains an open, untested hypothesis.

1 INTRODUCTION

One of the central goals of computational genomics is to predict biological function directly from DNA sequences. This goal encompasses tasks ranging from identifying functional elements, such as promoters, enhancers, transcription factor binding sites, and splice sites; to chromatin accessibility, gene expression, or estimating the functional effect of genetic variants.

To address this diversity of problems, DNA foundational models (FMs) have been developed in recent years. These models are pretrained using self-supervised objectives on large collections of genomes to learn generalizable features, which can then be adapted to different tasks through fine-tuning.

Transformer-based and SSM-inspired architectures have become the dominant approach in DNA modeling and FMs. Recently, the use of CNNs, which were the dominant approach before transformers, has also been re-explored in ConvNova (Bo et al., 2025). See section 2.1 for more details.

One of the main challenges of FMs is that DNA consists of two strands, each is the reverse complement (RC) of the other. Because a double-stranded sequence and its reverse complement describe the same physical molecule, a model that can't recognize this equivalence is wasting capacity relearning it separately for both orientations. We refer to this equivalence, that S, a DNA sequence and $RC(S)$, the RC of a DNA sequence, encode the same physical molecule, as DNA's RC-symmetry.

One of the main challenges of DNA modeling is designing the model architecture to exploit this RC-symmetry. It has been shown that making the model treat the RC on its input in an invariant or equivariant way improves performance as seen in Zhou et al. (2021) or Caduceus (Schiff et al., 2024), and it has been a leading practice for FMs since. See section 2.2 for more details.

But, even though $RC(S)$ is a deterministic and reversible transformation of S (knowing S implies knowing $RC(S)$), this does not imply that these strands are functionally equivalent. Moeckel et al. (2023) show functional asymmetries which are caused by several biological processes. Several of these asymmetries are associated with transcription, of which a few examples are (reviewed from Moeckel et al. (2023)):

- A GC-skew between the template and the coding strands, (Green et al., 2003), which in turn favors the formation of structures such as R-loops and G-quadruplexes (Ginno et al. (2012), Mao et al. (2018))
- Repetitive elements show significant transcriptional asymmetry (Georgakopoulos-Soares et al., 2021).
- Certain elements, such as promoters, are only functional when correctly oriented (Duttke et al., 2015).

Based on the available literature, few works have been found that incorporate the asymmetries shown above (see section 2.5). Most FMs focus on either:

a Processing both DNA strands in a fully symmetric way, as in RCPS, conjoined models (Zhou et al., 2021) and Caduceus (Schiff et al., 2024)

b Not explicitly incorporating this symmetry into the model architecture and instead letting the model learn it independently, as in Nucleotide Transformer (Dalla-Torre et al., 2025), HyenaDNA (Nguyen et al., 2023) and Zhou's RC-aug models (Zhou et al., 2021)

Although models that don't incorporate this RC-symmetry into their architectures could theoretically learn it, and even be capable of accounting for the asymmetries observed in Moeckel et al. (2023), better results can be observed when the RC-symmetry is incorporated explicitly into the architecture, as it can be seen in the decreased performance of Zhou et al. (2021) RC-aug models (see section 2.2) or the general improvement of Caduceus compared to HyenaDNA (Schiff et al., 2024).

However, the best-performing model does not fit either of the previous two categories. ConvNova outperforms Caduceus, the second-best model, on several tasks despite having significantly fewer parameters(Bo et al., 2025) and despite CNNs being generally considered surpassed by transformer and SSM-inspired architectures.

It doesn't force full RC-symmetry, but it doesn't ignore it either. Its symmetry-aware, by taking both strands as inputs, but lets the network modulate the symmetry on its own instead of forcing it through weight-tying.

This suggests that neither ignoring symmetry nor fully forcing it is optimal, and provides grounds for exploring an intermediate direction: being symmetry-aware without forcing invariance or equivariance.

Although RC-symmetric models avoid relearning each strand separately, but it still receives the two strands in an orientation that is arbitrary with respect to transcription. In the same input position, it will sometimes receive the coding strand, and other times the template strand. As a result, it cannot exploit the transcription-linked asymmetries described above, since the model's filters would have to consistently work well with template and coding strands, implicitly forcing the model to be RC-invariant to transcriptional asymmetries.

This suggests a next step to explore: Pushing this semi-symmetrical architecture further by making the model always receive both DNA strands in a consistent orientation by making the template and coding strands always occupy predefined positions within the input.

Passing both strands in this *orientation-consistent processing* would allow the model to capture the asymmetries that occur only in that specific strand, since if it were simply given both strands in either of the 2 orientations, the model would be forced to learn the biological signals that occur equally on both strands (and become RC-invariant with respect to transcriptional asymmetries).

This orientation-consistent concept could theoretically be applied to any architecture (it just wouldn't make sense in the case of an RC-equivariant or RC-invariant architecture, as it would have no effect).

This orientation-consistent learning can be achieved through a canonicalization function. Kaba et al. (2023) successfully uses canonicalization for a wide range of domains and proposes that the canonicalization function should be learned rather than hand-engineered.

Based on this, we present a proof of concept of this canonicalization, in which:

1. For each dataset, genome annotation data is obtained to determine which parts of the sequences are transcribed and which are the template/coding strands
2. To assess the effect of canonicalization, we train ConvNova, which is symmetry-aware without imposing RC invariance or equivariance (see section 3.2) on data from the Nucleotide Transformer benchmark(Dalla-Torre et al., 2025), Genomic Benchmark(Grešová et al., 2023), BEND Benchmark(Marin et al., 2023), and on chromatin profile prediction using the DeepSEA dataset(Zhou & Troyanskaya, 2015).
3. Multiple versions of the canonicalization algorithm are evaluated and compared.

The genome annotation data is obtained through GENCODE's General Feature Format (GFF) file, and since it constitutes the most reliable source available for inferring transcriptional orientation, these results should be interpreted as an upper bound on what is achievable for transcriptional asymmetry through a learned function.

2 Related work

2.1 CNNs, Transformers, and SSM-inspired architectures

CNNs such as Basset(Kelley et al., 2016), Basenji(Kelley et al., 2018), and LegNet(Penzar et al., 2023) were the dominant approach in DNA modeling before the rise of transformer-based architectures.

In recent years, transformer-based architectures such as Nucleotide Transformer (Dalla-Torre et al., 2025), Enformer (Avsec et al., 2021), DNABERT-2 (Zhou et al., 2024) have become the dominant approach as FMs thanks to attention's ability to deal with long-range dependencies and focus on specific signals.

Recently, SSM-inspired architectures such as HyenaDNA (Nguyen et al., 2023) and Caduceus (Schiff et al., 2024) have become popular because of their ability to capture long-range dependencies without the large computational cost of transformer-based architectures.

2.2 RC-aug, RCPS, and conjoined models

In Zhou et al. (2021), 3 types of CNNs are evaluated on predicting 919 chromatin features (TF binding, histone marks, and DNase I hypersensitivity). Each model takes a single DNA sequence as input, represented as a matrix where each row is the nucleotide base and each column is the nucleotide position. The models evaluated were:

- RC-augmented/RC-aug models: Data augmentation was applied by taking the RC of the sequence and keeping the target value, and the model was trained normally
- RCPS (Reverse Complement Parameter Sharing) models: The network's filters were designed so that $f(S) = f(RC(S))$ through weight tying.
- Conjoined (CJ) and conjoined post-hoc models: Both S and $RC(S)$ are passed separately through the same network, and their outputs are averaged. In CJ models, this averaging is applied during both training and inference, whereas in post-hoc models it is applied only at inference.

The results showed that the Conjoined post-hoc model were the best on the majority of tasks, with RCPS occasionally having a better performance than it.

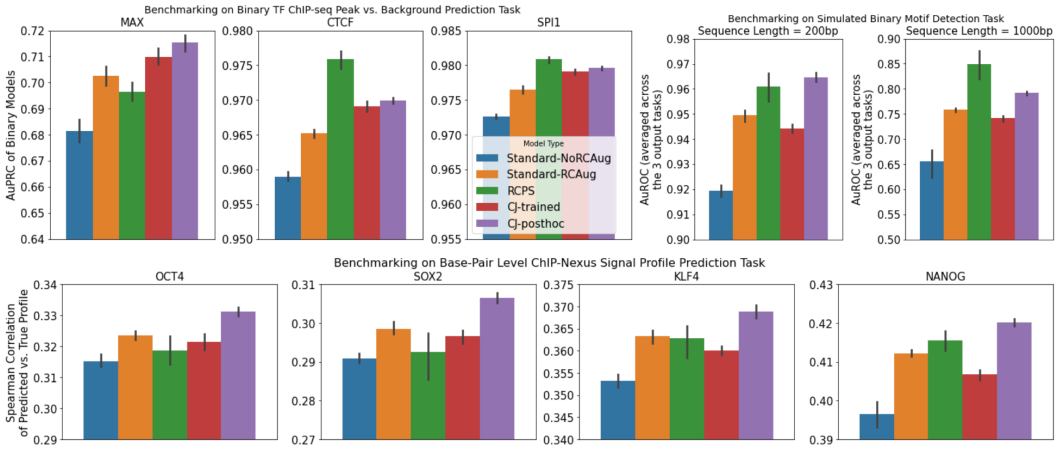


Figure 1: The RC-augmented(RC-aug), RCPS, conjoined (CJ-trained), and conjoined post-hoc (CJ-posthoc) models were evaluated on ChIP-seq, ChIP-nexus, and simulated datasets. Figure adapted from Zhou et al. (2021)

2.3 CANONICALIZATION

Canonicalization offers a distinct strategy for handling symmetries, distinct from both data augmentation and architectural weight-tying. Instead of forcing invariance within the network (as in RCPS) or implicitly teaching it through the data (as in RC-aug), canonicalization transforms the input to a standard orientation before it enters the model so that the network never needs to explicitly learn or impose the symmetry.

Kaba et al. (2023) formalizes canonicalization in a generalizable way for any kind of task, by making the model $\phi(x)$ be:

$$\phi(x) = h'(x) \cdot f\left(h^{-1}(x) \cdot x\right)$$

Where:

- $h(x)$ is the canonicalization function: it predicts the transformation that the canonical version of x underwent, and its inverse is taken to bring x to its canonical version
- $f(x)$ is the prediction function
- $h'(x)$ is the label canonicalization function: it predicts the transformation that the y of that canonical x underwent. On RC-invariant tasks, $h'(x)$ is the identity function for all x

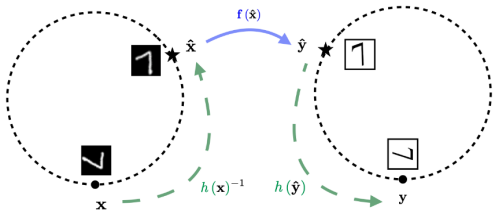


Figure 2: A diagram of the canonicalization process for a task where the goal was to recognize MNIST digits' rotation in an equivariant way, where the digit is transformed to its canonical form, recognized, and then the prediction is rotated back to its original form. Figure adapted from Kaba et al. (2023)

Its important to note that we use only the general concept of canonicalization: transforming the input to a standard orientation before processing it. It does not adopt the specific techniques that Kaba et al. (2023) propose to make this function learnable end-to-end.

2.4 ConvNova

ConvNova (Bo et al., 2025) is a convolutional network that was evaluated on multiple benchmarks: the Nucleotide Transformer benchmark(Dalla-Torre et al., 2025), Genomic Benchmark (Grešová et al., 2023), BEND Benchmark (Marin et al., 2023), and on chromatin profile prediction using the DeepSEA dataset (Zhou & Troyanskaya, 2015), and it outperformed many SoTA models despite having fewer parameters and a lower runtime.

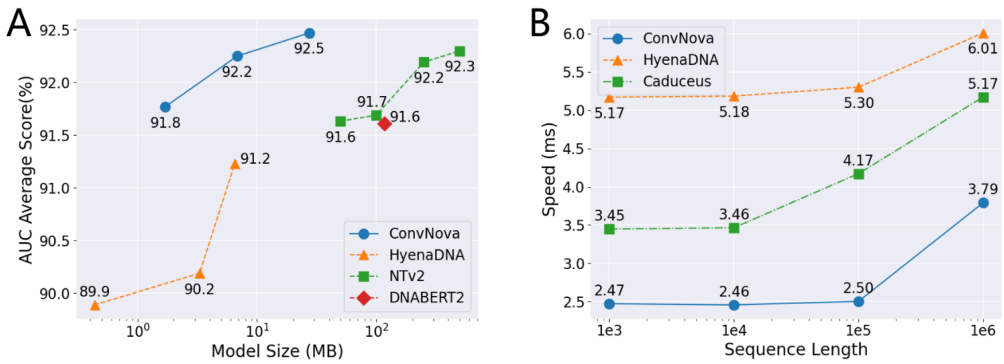


Figure 3: Trade-off between model performance and size for ConvNova and baseline methods. (A) Performance vs parameter count. (B) Inference time vs sequence length. Figure adapted from Bo et al. (2025)

2.5 Related work that deal with transcriptional asymmetry

Based on the available literature, the only works found that deal with the transcription-related asymmetry mentioned previously are:

- LegNet(Penzar et al., 2023), which adds an input channel `is_reverse` to the sequence indicating how to deal with the orientation-related asymmetries in regulatory elements.
- Asymmetron(Georgakopoulos-Soares et al., 2021), a toolkit for statistically analysing strand asymmetries.

3 Methods

3.1 Canonicalization

TODO: ACTUALIZAR CON LOS NUEVOS RESULTADOS DE HPO

The canonicalization method consists of a pre-processing pipeline that makes all the datasets orientation-consistent. It works in the following way:

1. Extract all transcribed features, pseudogenes, and V(D)J segments (in the case of human and mouse genomes), their positions, and whether they are oriented in coding or template form in the dataset. The orientation of the features is obtained directly from the 'Strand' field from the GFF.
   The way that features are classified as transcribed or not is: For annotations without a standardized type field (NCBI), the Sequence Ontology hierarchy (Eilbeck et al., 2005) was used to programmatically determine which features correspond to transcribed units; for GENCODE annotations, the 'Feature' and 'Transcript type' fields were directly used.
2. Based on that, the goal is for all of those selected features (we'll call these *segments* from now on) to be in their coding orientation by taking the reverse-complement of their coordinates in the genome.

3. In the case where there are overlaps between segments of opposite orientations, applying RC to one segment is considered to **completely** destroy both segments (this method produced the best results, see section 3.5 for more details). In that case, both segments are left as is.
4. After the whole genome is canonicalized, it's then saved and the model is trained on it

Multiple versions of this canonicalization algorithm were evaluated. For more details on the different algorithms, see section 3.5, and section 3.3 for the evaluation protocol.

For the downstream tasks, we need to know the sequence's position in the genome it was obtained from — referred to as the *source genome* — to know if it's located on top of a feature selected by the canonicalization algorithm and if it should be canonicalized. Most datasets don't provide the sequence's coordinates, and in those cases, they were obtained by aligning the sequences with their respective source genome using minimap2 (Li, 2018)

Then, after getting the sequence coordinates, if more than 50% of the sequence is transcribed, then the entire sequence is canonicalized along with its label. The basis of this algorithmic decision is based off the results from 3.3

The label transformation method depends on the tasks, which are divided into 3 categories:

- RC-invariant tasks where $f(RC(S)) = f(S)$, such as ChIP-seq results, for example (since it analyzes interactions on dsDNA)
- RC-equivariant tasks where $f(RC(S)) = T(f(S))$, with T being a known transformation over the output space. For RC-invariant tasks, $T = I$, the identity function.
- Non-canonicalizable tasks where $f(S) \neq f(RC(S))$, and there isn't a known T transformation. For example, in a promoter classification task, the RC of a sequence labelled as non-promoter might or might not correspond to a promoter. Therefore, the transformation of the negative class under RC cannot be determined solely from its original label. In these cases, canonicalization is skipped, because transforming the sequence would break its label.

For downstream tasks, regions without transcribed segments remain unmodified and passed in arbitrarily as they were on the source genome, as done in Bo et al. (2025). Even if the sequence falls on the negative strand and the model never saw it, the sequence won't be transformed in order to test the model's generalization.

It should also be noted that for the pretraining dataset, not all transcribed segments are canonicalized, since when there are overlaps, the canonicalization algorithm has to make compromises in certain places. For the downstream tasks though, it is assumed that all transcribed segments were canonicalized, since the goal is to evaluate how well the model learns to capture transcriptional asymmetries, not the compromises the canonicalization algorithm had to make.

3.2 NETWORK ARCHITECTURE

ConvNova was used for the evaluation of the canonicalization's performance by training and fine-tuning it on the canonicalized dataset. The network architecture is a CNN consisting of a series of Gated Convolution Blocks (GCBs), which consist of a gated convolution (Yu et al., 2019) with dilated filters (Yu & Koltun, 2016), with an MLP per token or over the whole pooled sequence at the end depending on the task.

The model takes the DNA and its reverse-complement as inputs A and B and computes the output as follows, updating the features of A depending on the gating imposed by the outputs of B

$$h = \text{GELU}(\text{convolution}(\text{A}))$$
$$g = \text{sigmoid}(\text{convolution}(\text{B}))$$
$$\text{A} = \text{A} + h \odot g$$
$$\text{B} = \text{B} + g$$

The dilation rate of the GCBs follows the sequence $[1, 1, d, d^2, d^3, \dots]$ for each block. After the last GCB, the masked output of A passed through and MLP, and then each token is passed through a linear layer, or the whole sequence is pooled and passed to a linear head depending on the task.

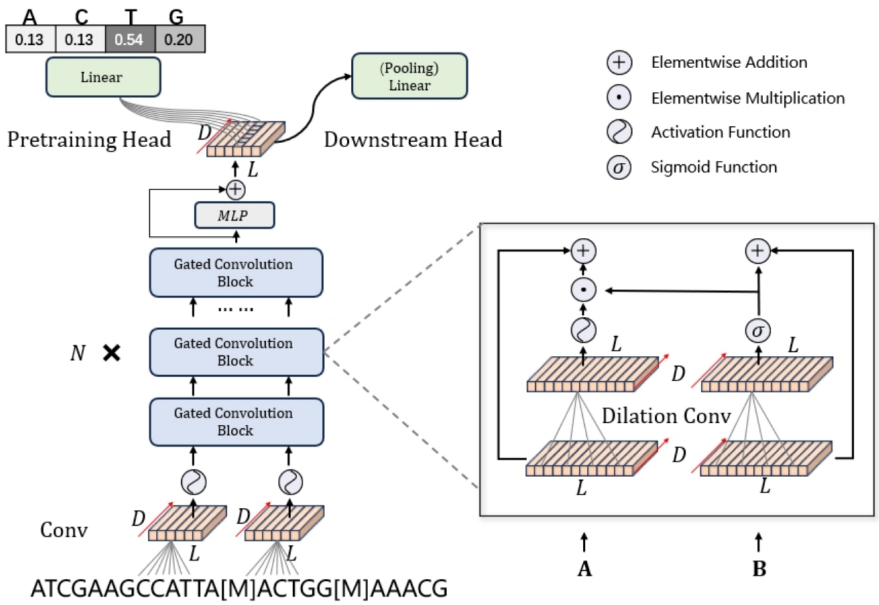


Figure 4: Diagram of ConvNova's architecture. Figure adapted from Bo et al. (2025)

ConvNova was chosen for three reasons: it achieves state-of-the-art metrics with a relatively small number of parameters; it is neither RC-invariant nor RC-equivariant, which would nullify the effect of canonicalization; and it takes both DNA strands as input, so it could potentially better learn the asymmetries between the forward and RC strands mentioned earlier.

3.3 Evaluation of the canonicalization algorithm

To evaluate the canonicalization method the training data is pre-processed (described in 3.1) and the same pretraining and fine-tuning protocol in Bo et al. (2025) paper is used (except for the minority cases where it is explicitly stated that a different protocol was used)

For each of the tasks, as with ConvNova, the model is pretrained on hg38 using masked language modelling(MLM). 15% of the nucleotides are masked and the model's objective is to predict them.

For the downstream tasks, simply comparing the performance of the non-canonicalized model vs the canonicalized one isn't enough. Because the canonicalization algorithm uses additional data, it possibly introduces a data leakage that could be inflating the true improvement caused by the canonicalization algorithm.

In the gene finding task from the BEND Benchmark, for example, a task that consists of classifying each nucleotide as exon, intron, acceptor, donor, or nothing, the (noisy) introduction of the strand orientation data by the canonicalization algorithm produces a leakage.

To mitigate that, an ablation was designed where 3 different versions of the model are evaluated:

- Baseline: The baseline model without any canonicalization
- Canonicalized: The model evaluated on the canonicalized datasets
- Leakage control: The model is evaluated on the non-canonicalized datasets with an additional input channel indicating whether that nucleotide is a positively transcribed feature (+1), a negatively transcribed feature (-1), or isn't transcribed (0)

The evaluation protocol for a model consists of 5 runs with 5 different random seeds. Those 3 versions of the model are evaluated on all downstream tasks using a fewer-parameter configuration to speed up the evaluation (see section A.2.3 for the configuration details).

Then, for the tasks where a promising difference appears between the canonicalized model vs the baseline and leakage control models, that same evaluation protocol is used but with the original, larger configuration. This screening step is only used to decide which tasks deserve the additional compute of a full-scale evaluation.

The criterion for deciding if an observed difference between the canonicalized model and the other two models is promising is by computing the right-tailed p-values for the canonicalized model vs the baseline, and the canonicalized model vs the leakage control one, taking the largest of those p-values, and checking if they're low enough.

It's worth noting that for the leakage control model, because of the additional input channel, has slightly more parameters than the baseline and canonicalized models. The parameter count difference is under 0.5% in relative terms and is therefore negligible.

3.4 DOWNSTREAM TASKS

For the choice of baselines and evaluation protocol, the original protocol established by the authors of ConvNova (Bo et al., 2025) will be followed, unless stated otherwise. The reason this protocol was used is that, by using ConvNova as the architecture and only changing the data pre-processing, the results are directly comparable.

All models were fine-tuned after being pretrained on hg38. For details regarding how the labels of each downstream task were canonicalized, see section A.3. The downstream tasks and their details are listed below. Each benchmark is evaluated a specific model configuration, see section A.2.2 for more details.

3.4.1 NT BENCHMARK

This benchmark is the one used by Nucleotide Transformer (Dalla-Torre et al. (2025)). It consists of 3 types of tasks: histone marker, splice site annotation and regulatory annotation prediction. It should be noted that the most recent version of this benchmark was used, which wasn't the one used in the original ConvNova paper.

To evaluate the tasks, the same methodology as in HyenaDNA (Nguyen et al., 2023) was followed, using the Matthews Correlation Coefficient (MCC) for certain tasks and the F1-score for others.

For canonicalization, GRCh38 was used as the source genome and the annotations were obtained from the Genome Reference Consortium, since all the datasets come from human samples. The tasks were evaluated using a pooling downstream head.

3.4.2 GENOMIC BENCHMARK

This is a multi-species benchmark, consisting of human, mouse, roundworm, and fruit fly samples. Its tasks consist of species classification, enhancer and regulatory annotation prediction.

For canonicalization, the following source genomes were used: hg38 for the human sample tasks, GRCm39 from the Genome Reference Consortium for the mouse tasks, and for the roundworm and fruit fly tasks the NCBI reference genomes were used. The tasks were evaluated using a pooling downstream head.

Note that for `human_ensembl_regulatory`, the baseline model's performance dropped compared to the original ConvNova paper because the whole sequence is passed to the model instead of a trimmed version of it like in the original ConvNova paper (see more details in section A.2.2). It's theorized that this happens because of the pooling head diluting the signal located at the start of the sequence used for prediction. This choice was made because it isn't realistic to have the biological signal consistently located at the start of the sequence.

3.4.3 BEND BENCHMARK

The model was evaluated on two tasks. gene finding, where each nucleotide of a DNA sequence is classified as donor, acceptor, intron, exon, or neither, along with the strand it lies on, and CpG

methylation, where given a DNA sequence multilabel classification is done on whether it is methylated or not in different cell lines.

The chromatin accessibility and histone modification tasks were considered redundant when comparing them to the rest of the other benchmarks' tasks, and therefore were excluded from the evaluation

The zero-shot variant effects tasks were also skipped, since unlike models with an attention mechanism, each output token is determined by a very small local context, and if one wanted to mitigate that through pooling, the SNP token signal becomes too diluted for classification. Evaluating those tasks with the original zero-shot protocol gives artificially low results.

For CpG methylation, the encoded sequence is pooled and passed through a linear head. For gene finding, each token is passed through a linear head.

3.4.4 Chromatin Profile Prediction

This task (Zhou & Troyanskaya, 2015) consists of predicting chromatin profiles and epigenetic markers given a DNA sequence. The dataset consists of 919 chromatin features, grouped into transcription factor binding profiles, DNase I hypersensitivity, and histone marks.

Even though the dataset's sequences were sourced from hg19, in this specific case, for canonicalization, hg38 is used as the source genome in order to use the most up-to-date annotations, and the coordinates are resolved directly by aligning the DeepSEA sequences with minimap2 (Li, 2018).

3.5 Optimal canonicalization parameters search

=== TODO=== ACTUALIZAR CON LOS NUEVOS PARAMETROS. TODO LO QUE HAGO DEBERIA SER EXPLICADO

The canonicalization pipeline and its parameters were optimized using Optuna (Akiba et al., 2019) using its default TPE optimizer. 20 trials were run to optimize the following parameters:

- `annotations_level`: Filter the GENCODE annotations to use up to level 1 (validated), 2 (manually annotated), or 3(automatically annotated).
- `transformation`: Whether to canonicalize a sequence by taking the RC, the complement, or the reverse of it.
- `overlap_transformation_behavior`: When encountering a segment **A** that overlaps with another segment **B** in the opposite orientation, this variable controls the expected behavior of transforming **A**:
  1. If `overlap_transformation_behavior` is set to `partial break`, then it's believed that only the overlapping part between **A** and **B** breaks
  2. If `overlap_transformation_behavior` is set to `full break`, then it's believed that the *whole* **B** segment breaks. It's behaviour depends on other variables specific to this.
  3. If `overlap_transformation_behavior` is set to `both break`, then it's believed that both **A** *and* **B** break. The algorithm chooses to either leave the sequence as-is(and leave **B** badly oriented) or to take the RC of the whole sequence(and leave **A** badly oriented) depending on which one is the longest segment

  See section A.1 for more details on the variables used and a more detailed explanation on the different canonicalization algorithms and the way they deal with overlapping segments.
- `include_cdjv_segments`: Whether to count the constant and V(D)J segments of the human and mouse adaptive immune system as features that should induce canonicalization or not.
- `include_pseudogenes`: Whether to count pseudogenes or not as features that should induce canonicalization.

See section 4.2 for the values chosen for these hyperparameters and more details on the HPO results.

20 trials were used, even though the number of possible configurations (180) is much larger than the 20 trials used, most of the hyperparameters had very little importance, and the improvements were starting to become marginal. See section 4.2 for more details on the results. It should also be noted that some of the variables are conditioned by others (see section A.1).

To speed up the trials, the model was pretrained and fine-tuned with a single configuration and the same pretraining weights for every downstream task, in order to obtain representative metrics for the configuration without having to use a different pretraining run per benchmark. See section A.2.3 for more details on the model configuration.

Each trial was evaluated on all downstream tasks, and its score was computed by averaging the z-score across those tasks. The mean and variance for those z-scores were obtained from the 5 baseline runs mentioned previously on section 3.3.

Even though there is a considerable task-type imbalance (there are many more histone-marks-related tasks than splicing-related tasks, for example), because there is little to no correlation between tasks, the score for each trial was obtained by simply averaging the z-scores for all tasks.

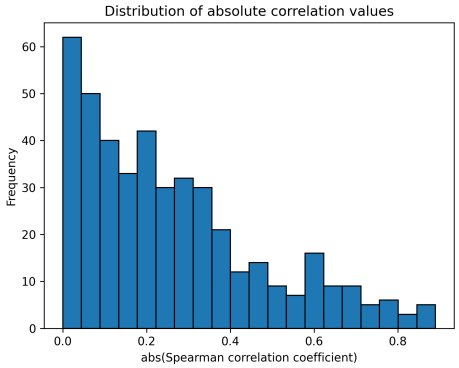


Figure 5: Histogram of the absolute values of the upper triangular section (without including the diagonal) of the Spearman correlation matrix of the downstream tasks.

4 Experiments

4.1 Comparison between the baseline, canonicalized and leakage control models

As it can be seen on figures 6 and 7, barely any tasks show a statistically promising difference, and even among those that do the effect size is weak, except for `demo_human_or_worm`. Still, it was hypothesized that the effect size between tasks may increase when evaluated on the full-scale model. The tasks chosen were: `H3K4me3`, `H4K20me1`, `enhancers`, `enhancers_types` and `demo_human_or_worm`. The results can be seen on table 1.

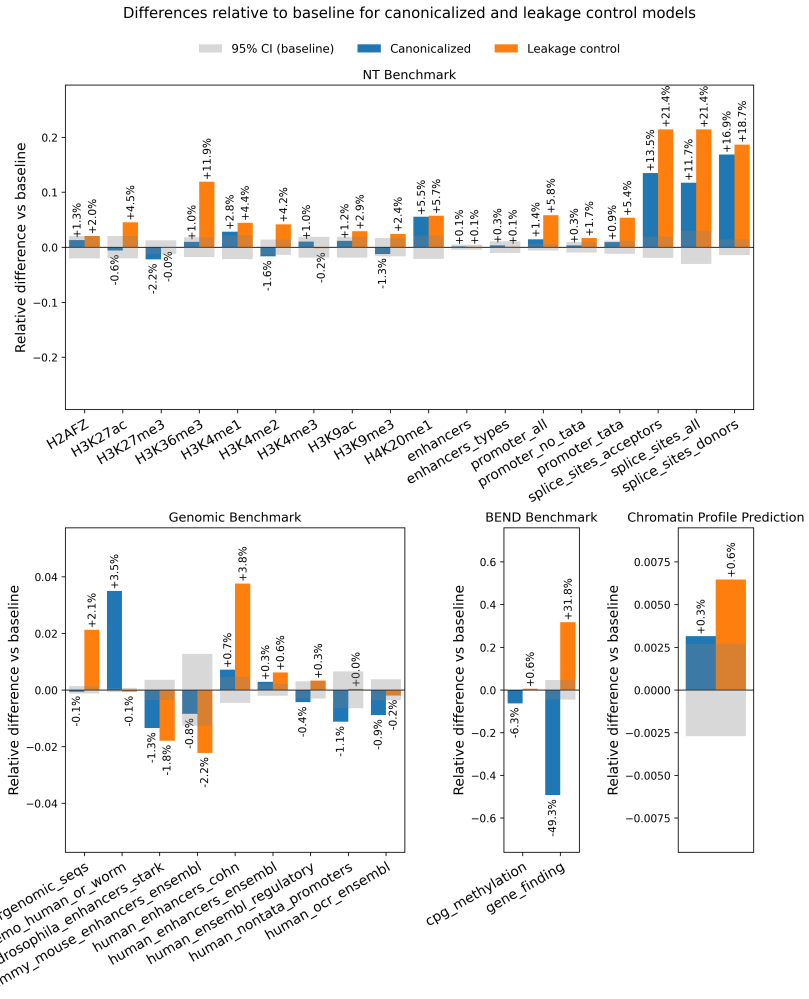


Figure 6: The canonicalized and leakage control mean values with respect to the baseline. The gray area around the baseline represents the 95% confidence interval for that task. Even though the canonicalized method produces better results than the baseline for most tasks, in the large majority of them, that effect seems to be produced by the data leakage mentioned previously (see section 3.3) because of the considerable difference between the canonicalized and leakage control methods

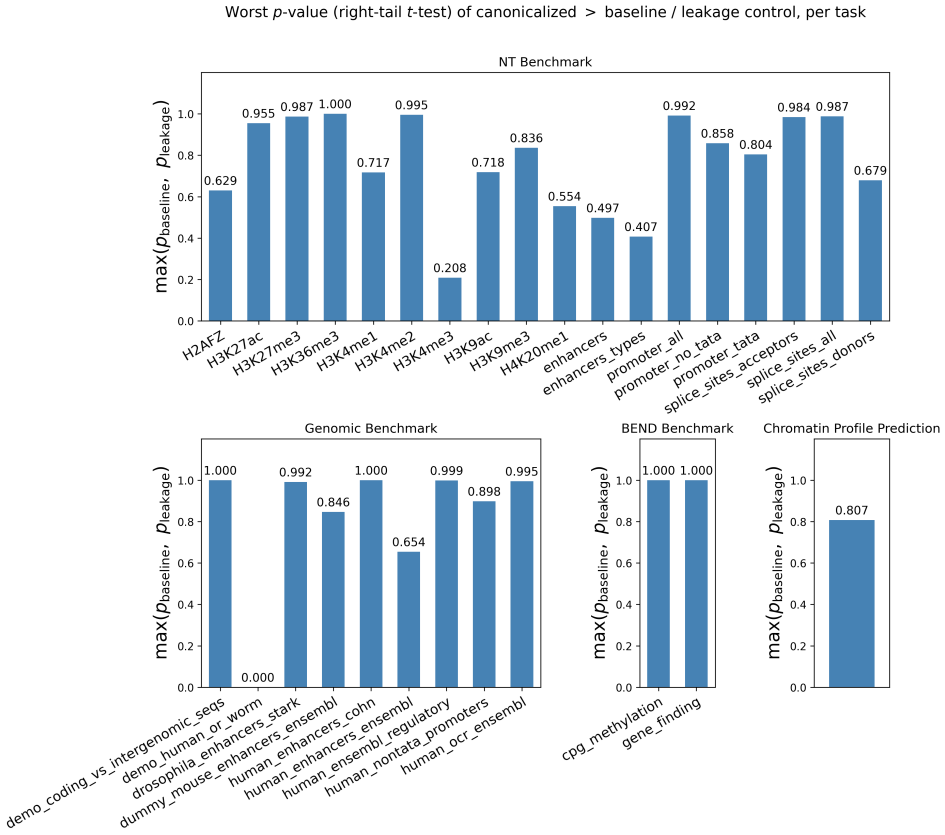


Figure 7: $\max(p_{\text{baseline}}, p_{\text{leakage control}})$ per task grouped by benchmark. Both p-values are computed using a right-tailed t-test between the canonicalized having a greater mean than the baseline and leakage control models

Dataset	Baseline	Canonicalized	Leakage Control	$\max(p_{\text{baseline}},\ p_{\text{leakage}})$
H3K4me3	0.607 ± 0.006	0.603 ± 0.010	0.617 ± 0.015	0.93
H4K20me1	0.571 ± 0.012	0.588 ± 0.007	0.596 ± 0.005	0.96
enhancers	0.500 ± 0.008	0.492 ± 0.004	0.511 ± 0.012	0.99
enhancers_types	0.458 ± 0.005	0.452 ± 0.007	0.469 ± 0.010	0.99
demo_human_or_worm	0.964 ± 0.001	0.965 ± 0.002	0.962 ± 0.003	0.29

Table 1: Summary statistics by dataset.

4.2 HYPERPARAMETER OPTIMIZATION RESULTS

===TODO ACTUALIZAR CON LOS NUEVOS RESULTADOS===

Out of the 20 trials ran, the optimal configuration for the hyperparameter optimization were:

- `annotations_level`: 2
- `transformation`: reverse-complement
- `overlap_transformation_behavior`: both break
- `include_pseudogenes`: true
- `include_cdjv_segments`: false

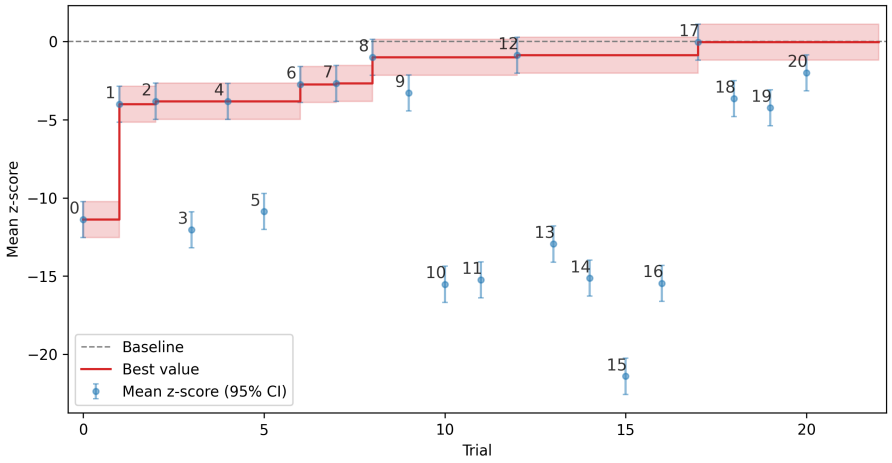


Figure 8: Optimization history showing the mean z-scores and their confidence interval for each trial

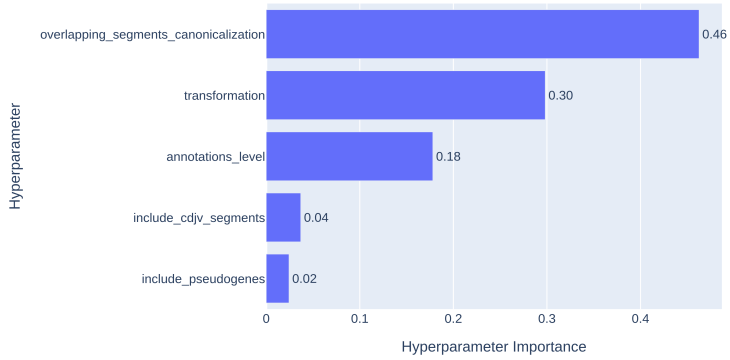


Figure 9: Hyperparameter importance plot produced by Optuna

5 Discussion

Even though the leakage control model has a higher score than the baseline and canonicalized model on almost all tasks, it can't be discerned whether it comes from the possible data leakage mentioned on section 3.3, or the exploitation of asymmetries. What is certain is that even if exploiting the asymmetry is beneficial, implementing that exploitation through canonicalization doesn't work for architectures like ConvNova.

It's noted that even though there could be a small improvement on the `demo_human_or_worm` task from Genomic Benchmark, it can't be known (assuming there's a true improvement) if it comes from the exploitation of transcriptional asymmetry, or through simpler reasons, like the difference in length, frequency and number of genes in human vs C. elegans genomes (Eichinger et al., 2005)

5.1 Future work

. Regarding future work, one could:

- Try to find another way to incorporate the transcriptional asymmetry into a model to assess whether this information can be exploited for better performance
- Modify the data used by the canonicalization method to evaluate it on the exploitation of the replicational asymmetries also mentioned by Moeckel et al. (2023)

- Evaluate the canonicalization method on a different architecture, like HyenaDNA(Nguyen et al., 2023)

REPRODUCIBILITY

The code for the project can be found on Github, with further guidance on how to reproduce the results there.

ACKNOWLEDGEMENTS

My thanks go to E. Matias Castro for valuable guidance on the research process.

REFERENCES

Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A Next-generation Hyperparameter Optimization Framework, July 2019.

Cosmas D. Arnold, Daniel Gerlach, Christoph Stelzer, Łukasz M. Boryń, and Al Et. Genome-Wide Quantitative Enhancer Activity Maps Identified by STARR-seq. *Science*, 2013. doi: 10.1126/science.1232542.

Preston R. Arnold, Andrew D. Wells, and Xian C. Li. Diversity and Emerging Roles of Enhancer RNA in Regulation of Gene Expression and Cell Fate. *Frontiers in Cell and Developmental Biology*, 7, January 2020. ISSN 2296-634X. doi: 10.3389/fcell.2019.00377.

Žiga Avsec, Vikram Agarwal, Daniel Visentin, Joseph R. Ledsam, Agnieszka Grabska-Barwinska, Kyle R. Taylor, Yannis Assael, John Jumper, Pushmeet Kohli, and David R. Kelley. Effective gene expression prediction from sequence by integrating long-range interactions. *Nature Methods*, 18 (10):1196–1203, October 2021. ISSN 1548-7105. doi: 10.1038/s41592-021-01252-x.

A. P. Bird. Use of restriction enzymes to study eukaryotic DNA methylation: II. The symmetry of methylated sites supports semi-conservative copying of the methylation pattern. *Journal of Molecular Biology*, 118(1):49–60, January 1978. ISSN 0022-2836. doi: 10.1016/0022-2836(78)90243-7.

Yu Bo, Weian Mao, Yanjun Shao, Weiqiang Bai, Peng Ye, Xinzhu Ma, Junbo Zhao, Hao Chen, and Chunhua Shen. Revisiting Convolution Architecture in the Realm of DNA Foundation Models. https://arxiv.org/abs/2502.18538v1, February 2025.

Laurel A. Coons, Adam B. Burkholder, Sylvia C. Hewitt, Donald P. McDonnell, and Kenneth S. Korach. Decoding the Inversion Symmetry Underlying Transcription Factor DNA-Binding Specificity and Functionality in the Genome. *iScience*, 15:552–591, May 2019. ISSN 25890042. doi: 10.1016/j.isci.2019.04.006.

Hugo Dalla-Torre, Liam Gonzalez, Javier Mendoza-Revilla, Nicolas Lopez Carranza, Adam Henryk Grzywaczewski, Francesco Oteri, Christian Dallago, Evan Trop, Bernardo P. de Almeida, Hassan Sirelkhatim, Guillaume Richard, Marcin Skwark, Karim Beguir, Marie Lopez, and Thomas Pierrot. Nucleotide Transformer: Building and evaluating robust foundation models for human genomics. *Nature Methods*, 22(2):287–297, February 2025. ISSN 1548-7105. doi: 10.1038/s41592-024-02523-z.

Sascha H. C. Duttke, Scott A. Lacadie, Mahmoud M. Ibrahim, Christopher K. Glass, David L. Corcoran, Christopher Benner, Sven Heinz, James T. Kadonaga, and Uwe Ohler. Human Promoters Are Intrinsically Directional. *Molecular Cell*, 57(4):674–684, February 2015. ISSN 1097-2765. doi: 10.1016/j.molcel.2014.12.029.

L. Eichinger, J. A. Pachebat, G. Glöckner, M.-A. Rajandream, R. Sucgang, M. Berriman, J. Song, R. Olsen, K. Szafranski, Q. Xu, B. Tunggal, S. Kummerfeld, M. Madera, B. A. Konfortov, F. Rivero, A. T. Bankier, R. Lehmann, N. Hamlin, R. Davies, P. Gaudet, P. Fey, K. Pilcher, G. Chen, D. Saunders, E. Sodergren, P. Davis, A. Kerhornou, X. Nie, N. Hall, C. Anjard, L. Hemphill, N. Bason, P. Farbrother, B. Desany, E. Just, T. Morio, R. Rost, C. Churcher, J. Cooper, S. Haydock,

N. Van Driessche, A. Cronin, I. Goodhead, D. Muzny, T. Mourier, A. Pain, M. Lu, D. Harper, R. Lindsay, H. Hauser, K. James, M. Quiles, M. Madan Babu, T. Saito, C. Buchrieser, A. Wardroper, M. Felder, M. Thangavelu, D. Johnson, A. Knights, H. Loulseged, K. Mungall, K. Oliver, C. Price, M. A. Quail, H. Urushihara, J. Hernandez, E. Rabbinowitsch, D. Steffen, M. Sanders, J. Ma, Y. Kohara, S. Sharp, M. Simmonds, S. Spiegler, A. Tivey, S. Sugano, B. White, D. Walker, J. Woodward, T. Winckler, Y. Tanaka, G. Shaulsky, M. Schleicher, G. Weinstock, A. Rosenthal, E. C. Cox, R. L. Chisholm, R. Gibbs, W. F. Loomis, M. Platzer, R. R. Kay, J. Williams, P. H. Dear, A. A. Noegel, B. Barrell, and A. Kuspa. The genome of the social amoeba Dictyostelium discoideum. *Nature*, 435(7038):43–57, May 2005. ISSN 0028-0836, 1476-4687. doi: 10.1038/nature03481.

Karen Eilbeck, Suzanna E. Lewis, Christopher J. Mungall, Mark Yandell, Lincoln Stein, Richard Durbin, and Michael Ashburner. The Sequence Ontology: A tool for the unification of genome annotations. *Genome Biology*, 6(5):R44, 2005. ISSN 1474-760X. doi: 10.1186/gb-2005-6-5-r44.

Ilias Georgakopoulos-Soares, Ioannis Mouratidis, Guillermo E. Parada, Navneet Matharu, Martin Hemberg, and Nadav Ahituv. Asymmetron: A toolkit for the identification of strand asymmetry patterns in biological sequences. *Nucleic Acids Research*, 49(1):e4, January 2021. ISSN 1362-4962. doi: 10.1093/nar/gkaa1052.

Paul A. Ginno, Paul L. Lott, Holly C. Christensen, Ian Korf, and Frédéric Chédin. R-loop formation is a distinctive characteristic of unmethylated human CpG island promoters. *Molecular Cell*, 45 (6):814–825, March 2012. ISSN 1097-4164. doi: 10.1016/j.molcel.2012.01.017.

Phil Green, Brent Ewing, Webb Miller, Pamela J. Thomas, NISC Comparative Sequencing Program, and Eric D. Green. Transcription-associated mutational asymmetry in mammalian evolution. *Nature Genetics*, 33(4):514–517, April 2003. ISSN 1061-4036. doi: 10.1038/ng1103.

Katarína Grešová, Vlastimil Martinek, David Čechák, Petr Šimeček, and Panagiotis Alexiou. Genomic benchmarks: A collection of datasets for genomic sequence classification. *BMC Genomic Data*, 24(1):25, May 2023. ISSN 2730-6844. doi: 10.1186/s12863-023-01123-8.

Max Jaderberg, Karen Simonyan, Andrew Zisserman, and koray kavukcuoglu. Spatial Transformer Networks. In *Advances in Neural Information Processing Systems*, volume 28. Curran Associates, Inc., 2015.

Sékou-Oumar Kaba, Arnab Kumar Mondal, Yan Zhang, Yoshua Bengio, and Siamak Ravanbakhsh. Equivariance with Learned Canonicalization Functions, July 2023.

Mira T. Kassouf, Helena S. Francis, Matthew Gosden, Maria C. Suciu, Damien J. Downes, Caroline Harrold, Martin Larke, Marieke Oudelaar, Lucy Cornell, Joseph Blayney, Jelena Telenius, Barbara Xella, Yuki Shen, Nikolaos Sousos, Jacqueline A. Sharpe, Jacqueline Sloane-Stanley, Andrew J. H. Smith, Christian Babbs, Jim R. Hughes, and Douglas R. Higgs. The α-globin super-enhancer acts in an orientation-dependent manner. *Nature Communications*, 16(1):1033, January 2025. ISSN 2041-1723. doi: 10.1038/s41467-025-56380-1.

David R. Kelley, Jasper Snoek, and John L. Rinn. Basset: Learning the regulatory code of the accessible genome with deep convolutional neural networks. *Genome Research*, 26(7):990–999, July 2016. ISSN 1088-9051. doi: 10.1101/gr.200535.115.

David R. Kelley, Yakir A. Reshef, Maxwell Bileschi, David Belanger, Cory Y. McLean, and Jasper Snoek. Sequential regulatory activity prediction across chromosomes with convolutional neural networks. *Genome Research*, 28(5):739–750, May 2018. ISSN 1088-9051. doi: 10.1101/gr.227819.117.

Heng Li. Minimap2: Pairwise alignment for nucleotide sequences. *Bioinformatics*, 34(18):3094–3100, September 2018. ISSN 1367-4803. doi: 10.1093/bioinformatics/bty191.

Shi-Qing Mao, Avazeh T. Ghanbarian, Jochen Spiegel, Sergio Martínez Cuesta, Dario Beraldi, Marco Di Antonio, Giovanni Marsico, Robert Hänsel-Hertsch, David Tannahill, and Shankar Balasubramanian. DNA G-quadruplex structures mold the DNA methylome. *Nature Structural & Molecular Biology*, 25(10):951–957, October 2018. ISSN 1545-9985. doi: 10.1038/s41594-018-0131-8.

Frederikke Isa Marin, Felix Teufel, Marc Horlacher, Dennis Madsen, Dennis Pultz, Ole Winther, and Wouter Boomsma. BEND: Benchmarking DNA Language Models on biologically meaningful tasks. https://arxiv.org/abs/2311.12570v4, November 2023.

Camille Moeckel, Apostolos Zaravinos, and Ilias Georgakopoulos-Soares. Strand asymmetries across genomic processes. *Computational and Structural Biotechnology Journal*, 21:2036–2047, March 2023. ISSN 2001-0370. doi: 10.1016/j.csbj.2023.03.007.

Eric Nguyen, Michael Poli, Marjan Faizi, Armin Thomas, Callum Birch-Sykes, Michael Wornow, Aman Patel, Clayton Rabideau, Stefano Massaroli, Yoshua Bengio, Stefano Ermon, Stephen A. Baccus, and Chris Ré. HyenaDNA: Long-Range Genomic Sequence Modeling at Single Nucleotide Resolution, November 2023.

Dmitry Penzar, Daria Nogina, Elizaveta Noskova, Arsenii Zinkevich, Georgy Meshcheryakov, Andrey Lando, Abdul Muntakim Rafi, Carl de Boer, and Ivan V. Kulakovskiy. LegNet: A best-in-class deep learning model for short DNA regulatory regions. *Bioinformatics*, 39(8):btad457, August 2023. ISSN 1367-4811. doi: 10.1093/bioinformatics/btad457.

Yair Schiff, Chia-Hsiang Kao, Aaron Gokaslan, Tri Dao, Albert Gu, and Volodymyr Kuleshov. Caduceus: Bi-Directional Equivariant Long-Range DNA Sequence Modeling, June 2024.

Fisher Yu and Vladlen Koltun. Multi-Scale Context Aggregation by Dilated Convolutions, April 2016.

Jiahui Yu, Zhe Lin, Jimei Yang, Xiaohui Shen, Xin Lu, and Thomas Huang. Free-Form Image Inpainting with Gated Convolution, October 2019.

Hannah Zhou, Avanti Shrikumar, and Anshul Kundaje. Towards a Better Understanding of Reverse-Complement Equivariance for Deep Learning Models in Regulatory Genomics, February 2021.

Jian Zhou and Olga G Troyanskaya. Predicting effects of noncoding variants with deep learning–based sequence model. *Nature methods*, 12(10):931–934, October 2015. ISSN 1548-7091. doi: 10.1038/nmeth.3547.

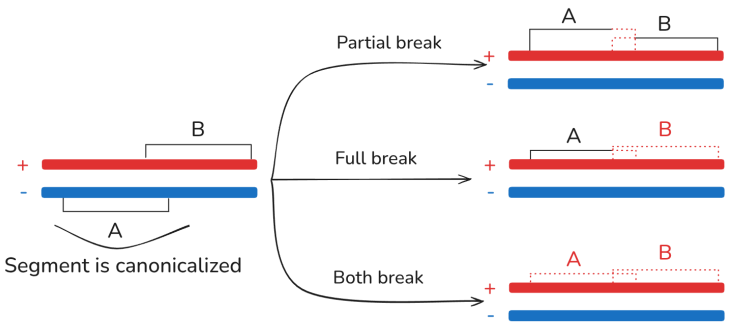
Zhihan Zhou, Yanrong Ji, Weijian Li, Pratik Dutta, Ramana Davuluri, and Han Liu. DNABERT-2: Efficient Foundation Model and Benchmark For Multi-Species Genome, March 2024.

A Appendix

A.1 Design of the canonicalization algorithm for overlapping segments

Regarding the approach to dealing with overlapping segments, 3 variables were proposed with which to formulate all the hypotheses of the optimal algorithm:

- `overlap_transformation_behavior`: Determines the different hypotheses for what happens when canonicalizing a segment **A** that overlaps with a segment **B**:
  - *Partial break*: Only the overlapping part between **A** and **B** of both segments breaks.
  - *Full break*: The entire segment **B** is broken when partially canonicalizing one of its parts.
  - *Both break*: Both **A** and **B** are entirely broken, since that overlapping part needed to be kept intact for both segments to be understood correctly by the model.



- `orient_all_segments`: A binary variable indicating whether it is necessary for all segments to be in the canonical orientation, or if the algorithm can ignore certain segments if the cost of ignoring them (the number of nucleotides) is lower than the cost of canonicalizing them (the number of nucleotides it destroys, which is calculated depending on `overlap_transformation_behavior`). The only case where it's irrelevant is when `overlap_transformation_behavior = partial_break`. This is explained in more detail below.
- `keep_segments_order`: In the case where `overlap_transformation_behavior = full_break` or `both_break` and `orient_all_segments=True` there are 2 ways to canonicalize the segments, one that breaks the fewest nucleotides but destroys the original order of the segments, and another where that original order is kept but more nucleotides are destroyed. This variable decides which of the 2 ways to use. They are described in detail on 2b

Based on all the possible permutations of these variables, the following canonicalization methods arise:

1. If `overlap_transformation_behavior=partial_break`, `orient_all_segments` (and `keep_segments_order`) have no effect since the cost of ignoring a segment is always $\geq$ the cost of canonicalizing it, because the length of the overlap is always less than or equal to the length of the segment, so all segments are oriented, regardless of whether there is an overlap or not

2. If `overlap_transformation_behavior=full_break`, there are 3 possible canonicalization algorithms. They all rely on building a *conflict window* first. When a negative segment that overlaps with a positive one is found, the set of the whole "chain" of overlapping segments is computed recursively. This set is defined as the conflict window.
   The reason conflict windows are used instead of simply looking at immediate overlaps, is that there are cases where only looking at immediate overlaps ends up in a bad decision. As shown in Figure 10, for example, if I decide whether to canonicalize **A** or not:
   a If I only check the immediate overlaps, then I do not canonicalize **A** since it is preferable to destroy it rather than **B** (because **B** is longer)
   b If I check the entire conflict window, it is preferable for me to canonicalize the negative segments (100bp) and destroy **B** rather than ignore them.

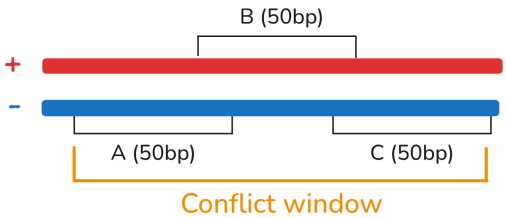


Figure 10: Example showing the reason behind the usage of conflict windows

After the conflict window is built, the canonicalization algorithm branches in:

(a) If `orient_all_segments=False`, then the cheapest option is chosen. Orient the negative segments at the cost of destroying the positive ones (the sum of the lengths of positive segments in the conflict window), or ignore the negative segments, at a cost equal to the sum of their lengths.

(b) If `orient_all_segments=True`, then there are 2 ways to deal with the overlapping segments:

i. If one wishes to keep the exact relative order of the segments as they were (`keep_segments_order=True`), then, across the whole genome, one can either canonicalize the segments from the - to the + strand in all of the conflict windows, or canonicalize the segments from the + to the - strand in all of them, and then take the RC of the entire genome to bring them all to the + strand.

Although the original absolute order of the segments is not maintained when applying the second option, relative order of "which one follows which" is still maintained. An example of this canonicalization method can be seen on Figure 11

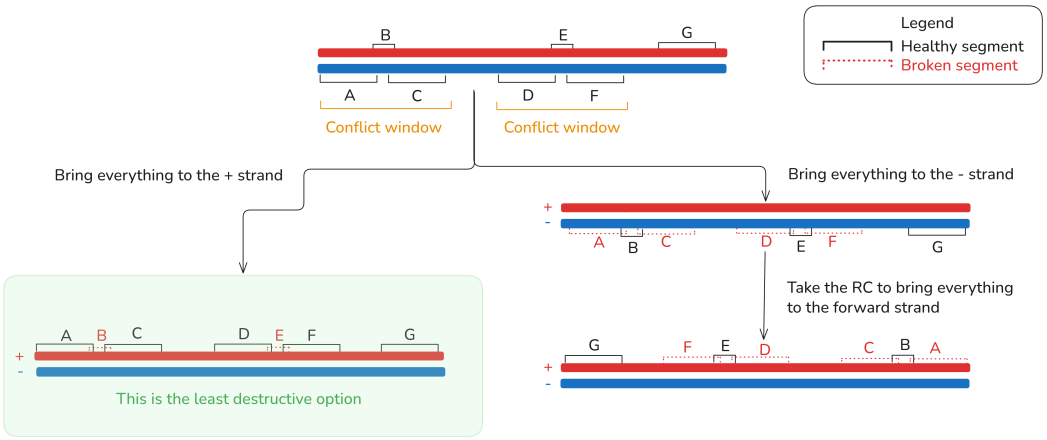


Figure 11

=== TODO. EN "SOME ARE REVERSED" ASEGURARME QUE LA TRANSFORMACION ELEGIDA POR EL HPO NUEVO SEA REVERSE O RC===

ii. If the canonicalization with the lowest cost is desired, regardless of the relative order of the segments, a procedure similar to the previous one is applied but scoped at the conflict window level instead of the whole chromosome level. The reason the order is lost is that some overlap windows are reversed and others are not (if the transformation isn't set to `complement`), causing the order to be lost at the genomic level. An example of this method can be seen below on Figure 12

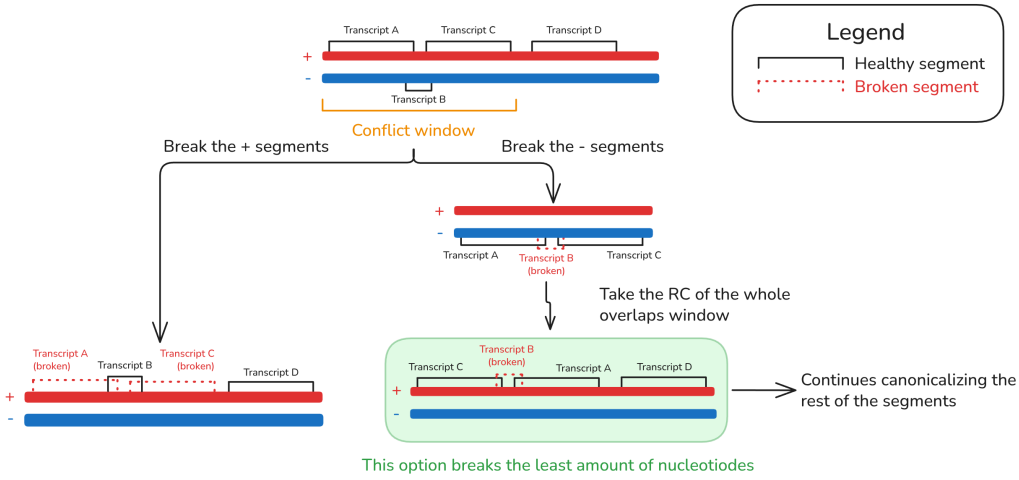


Figure 12

3. If `overlap_transformation_behavior=both_break`, there are two possible canonicalization algorithms. Both algorithms consist of canonicalizing the segments that don't overlap with any segments on the opposite strand, and for the ones that do, instead of canonicalizing them — which would always cost more, because the cost of canonicalizing them (length(**A**) + length(**B**)) is always higher than the cost of ignoring them (length(**A**)) — it's chosen whether to keep the positive strand of the segments or the negative one, depending on which strand has the most coding nucleotides.

   (a) If `keep_segments_order=True` the algorithm chooses whether to canonicalize all the non-overlapping negative segments in the chromosome, or to canonicalize all the non-overlapping positive segments and then take the RC of the whole chromosome

   (b) If `keep_segments_order=False`, the algorithm canonicalizes all the non-overlapping negative segments in the chromosome, and then, for each conflict window, it decides whether to leave it as-is or to canonicalize it.

   This algorithm is similar to the one used when `overlap_transformation_behavior=full_break` and `orient_all_segments=False`, the only difference is in that case, all the segments(including the ones that have overlaps) are canonicalized.

The advantages and disadvantages of the possible configurations of `keep_segments_order` can be seen on Figure 13.

- In the case where `keep_segments_order=True`, the canonicalization is done that way because destroying the *B*, *D* and *F* segments (110bp) is cheaper than destroying the *A*, *C* and *E* segments (210bp)

- In the case where `keep_segments_order=False` each overlap window is processed individually, and even though the result destroys the least amount of nucleotides, the relative order between coding sequences is lost (as seen in the second conflict window)

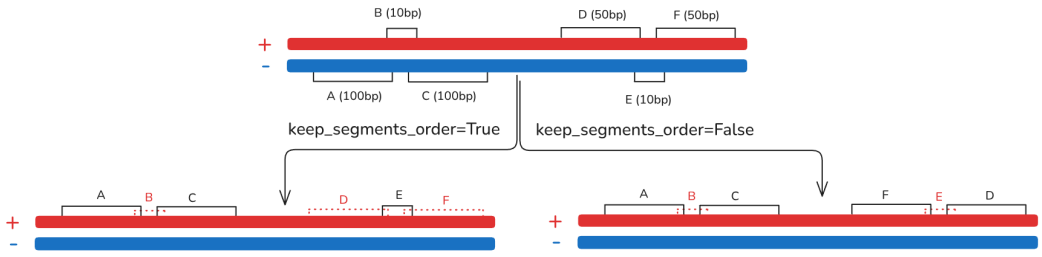
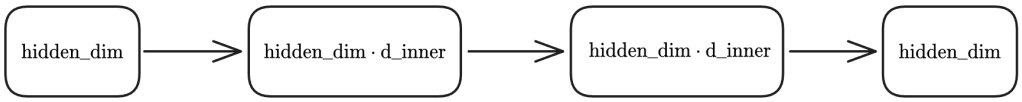


Figure 13

A.2 Model parameters

It's worth noting that `d_inner` determines the dimension of the middle layers of the MLP that comes before the downstream head, as seen on the figure below



A.2.1 Pretraining

The same configuration as the original ConvNova paper was used, which was similar to HyenaDNA.

Hyperparameter	Value
Learning Rate	$\beta_1 = 0.9, \beta_2 = 0.999$
Batch Size	256
Weight Decay	0.1
Dropout	0.0
Optimizer	AdamW
Optimizer Momentum	$\beta_1 = 0.9, \beta_2 = 0.999$
Learning Rate Scheduler	Cosine Decay
Epochs	400
Dilation rate	4
Precision	fp32
Kernel size	9

Different sequence lengths were used for each model:

Parameters	378k	1.7M	6.8M	7.4M
Sequence length	1k	1k	2k	16k

A.2.2 Downstream tasks

For the downstream tasks, the same configuration as for pretraining was used but with the following changes:

The configuration is similar to the one used in ConvNova, with a slight difference in the sequence length for Genomic Benchmark. While they trained ConvNova on Genomic Benchmark with a sequence length of 1k (to have a direct comparison against transformer-based models), some tasks on Genomic Benchmark have an actual higher sequence length (one of the tasks, for example, has a

Task	Sequence Length	Hidden dim	Num GCBs	Size	d_inner
NT Benchmark	Depends on the task, shown below	128	5	1.7M	2
Genomic Benchmark	Depends on the task, shown below	64t	5	378k	4
BEND Benchmark	512 for CpG methylation, 16k for gene finding	196	10	7.4M	2
Chromatin Profile Prediction	1k	256	5	6.8M	2

sequence length of 5k), and it was decided to use those true sequence lengths to have a more accurate representation of the canonicalization algorithm's performance.

Note that for the Genomic Benchmark, even though the original ConvNova paper reports a 386k parameter model for this task, the configurations provided in the paper and its GitHub repo don't match this parameter count. The authors were contacted with no response, so the configuration that comes closest to 386k parameters, a 378k one, was chosen.

Table 3: Sequence lengths for NT Benchmark

Task	Sequence length
H2AFZ	1000
H3K27ac	1000
H3K27me3	1000
H3K36me3	1000
H3K4me1	1000
H3K4me2	1000
H3K4me3	1000
H3K9ac	1000
H3K9me3	1000
H4K20me1	1000
promoter_all	300
promoter_tata	300
promoter_no_tata	300
splice_sites_all	600
splice_sites_donor	600
splice_sites_acceptor	600
enhancers	400
enhancers_types	400

Table 4: Sequence lengths for Genomic Benchmark

Task	Sequence length
dummy_mouse_enhancers_ensembl	5000
demo_coding_vs_intergenomic_seqs	200
demo_human_or_worm	200
human_enhancers_cohn	500
human_enhancers_ensembl	573
human_ensembl_regulatory	802
human_nontata_promoters	251
human_ocr_ensembl	600
drosophila_enhancers_stark	3250

A.2.3 HYPERPARAMETER SEARCH

For the hyperparameter search, the model was pretrained and fine-tuned on all tasks with the same pretraining and downstream tasks configuration, but with the following differences across all of them.

It should be noted that a single pretrained model per trial was used for the rest of the downstream tasks. The difference in precision was to speed up the hyperparameter search, *and the number of pretraining epochs was also changed from 400 to 200.*

The 5 random seeds used for the hyperparamter search were: 2222, 205, 8262, 1480 and 4436.

Hidden dim	Num GCBs	Kernel size	d_inner	Size	Precision	Random seed
64	5	5	2	262k	bf16	2222

A.3 CLASSIFICATION OF TASKS INTO RC-INVARIANT, RC-EQUIVARIANT, OR OMITTED

Many benchmark tasks can generally be classified into tasks of histone marks, enhancers, promoters, and splicing. These task classes were treated as specified in the table below.

It is worth mentioning that tasks where the canonicalization was omitted were still evaluated because it was hypothesized that the canonicalized pretraining may change the performance of the model on those tasks.

Table 5: **Classification and rationale for genomic tasks**

Task	Class	Reason
Histone marks tasks and OCR (open chromatin region) tasks	RC-invariant	Histones interact with double-stranded DNA, so their marks are invariant to the RC of the sequence. The same applies to OCR tasks.
Enhancer tasks	RC-invariant	Most enhancers are functionally bidirectional(Arnold et al., 2013), and in the majority of cases, both enhancer strands get transcribed into eRNA(Arnold et al., 2020). Even so, it should be noted that there are special cases where enhancers are orientation-dependent, like in super-enhancers (Kassouf et al., 2025).
Promoter tasks	Omitted	Promoters are inherently directional, since they only function on one of the strands — otherwise, both strands would be transcribed. There is no way to know without experimental data whether the RC of a promoter is a promoter or not, or whether the RC of a non-promoter sequence produces a promoter. That is why they were omitted.
Splicing tasks	Omitted	Applying the canonicalization to sequences that contain acceptor or donor sites should have no effect because the sequences passed are already on the positive coding strand. For the sequences without acceptor or donor sites, applying the canonicalization algorithm could invert the sequence's label if it falls on the template strand of a coding region, and I can't update the labels because I can't be certain of a sequence label without experimental data

For the particular cases, the way each of them is handled is detailed below:

A.3.1 NUCLEOTIDE TRANSFORMER BENCHMARK

There are no particular tasks that don't fit within the classes mentioned above.

A.3.2 GENOMIC BENCHMARK

The only tasks that don't fit within the classes mentioned above are:

Table 6: Classification and rationale for genomic tasks

Task	Class	Reason
demo_human_or_worm	RC-invariant	The RC of human DNA is still human DNA; the same applies to any other species.
demo_coding_vs_intergenomic_seqs	RC-invariant	Taking the RC of a sequence positioned on a transcribed feature does not move it outside that feature; it only changes its orientation. The same applies to non-transcribed sequences.
human_ensembl_regulatory	RC-invariant for enhancers and OCR; promoter samples are omitted	The task consists of classifying a DNA sequence among enhancers, promoters, and OCR. What each strand is is identified using the labels, and promoters are omitted.

A.3.3 BEND BENCHMARK

Table 8: Classification and rationale for BEND Benchmark tasks

Task	Class	Reason
Gene finding	RC-equivariant	The task consists of classifying each nucleotide as {exon forward, donor forward, intron forward, acceptor forward, exon reverse, ..., non-coding}. Taking the RC of a coding nucleotide changes its orientation and changes its position.
CpG Methylation	RC-invariant	The reverse complement of CG is CG, and they are methylated on both strands Bird (1978). And even though a `reverse` or `complement` would break the CG site by turning into a GC, the task won't be omitted to see the RC transformation behaviour on it.

A.3.4 CHROMATIN PROFILE PREDICTION

Most transcription factors (TFs) are functionally invariant to the reverse complement(Coons et al., 2019), so the task is treated as RC-invariant.